

Design Agent – Developer Guide

This guide explains how to use the `/design` agent within the MiDAS GitHub workflow to generate comprehensive architecture and system design documentation.

Introduction

The `/design` agent is a software architecture and system design expert integrated into the MiDAS GitHub workflow. It is triggered through GitHub issue comments and produces structured, detailed design documents in Markdown format.

The agent analyzes business requirements, technical constraints, scale considerations, and repository context to generate complete architecture specifications. The output includes system components, data models, API contracts, deployment strategies, technical decisions, and trade-off analysis.

This agent is intended to standardize design documentation across the organization and ensure architectural clarity before implementation begins.

What Are the Use Cases for the Design Agent

The `/design` agent should be used when structured architectural thinking is required before development begins.

Common Use Cases

- Designing a new subsystem or service
- Adding a major feature to an existing repository
- Refactoring architecture at a system level
- Evaluating technical approaches before implementation
- Creating formal design documentation for review



- Preparing architecture documentation for stakeholders

When to Use `/design` Instead of `/implement`

Use `/design` when:

- The problem statement is defined but the technical approach is not finalized.
- Multiple architectural options must be evaluated.
- The change impacts system components, APIs, or infrastructure.
- You require a formal design artifact before coding.

Do not use `/design` for small code fixes or minor enhancements.

How to Use the Design Agent

The `/design` agent is triggered from GitHub issue comments.

Step 1: Open or Navigate to a GitHub Issue

Ensure the issue clearly describes:

- Business requirement
- Functional expectations
- Constraints (if known)
- Existing context (if modifying an existing system)



Test Issue - Add Health Check Endpoint to SS6 Service #3768

 Open KES1HYD opened this issue 7 minutes ago · 0 comments

 KES1HYD commented 7 minutes ago

Description

We need to add a basic health check endpoint to the SS6 service. This endpoint should allow external systems and monitoring tools to verify that the service is running and responsive.

The endpoint should return a simple success response when the service is healthy.

The goal of this task is to design the endpoint structure, response format, and integration approach within the existing SS6 architecture.

Acceptance Criteria

- A `/health` or `/healthz` endpoint is defined.
- The endpoint returns HTTP 200 when the service is healthy.
- The response format is clearly defined (JSON).
- The design specifies where the endpoint will be implemented (router/controller).
- Any dependency checks (optional DB, cache, etc.) are defined.
- Security considerations are documented (public vs internal access).

Tasks

- Define API contract for health endpoint.
- Identify integration location in SS6 service.
- Define response structure.
- Document assumptions and constraints.

Step 2: Trigger the Agent

In the issue comment, type:

```
@MiDAS /design <your query>
```

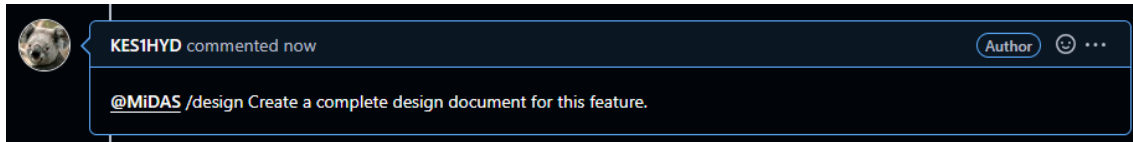
Example:

```
@MiDAS /design Create a design document for this repository.
```

Another example:

```
@MiDAS /design Design a scalable evaluation service for SS6 with authentication and rate limiting.
```





Step 3: Review the Generated Design Document

The agent will respond with a structured response that may include:

- Problem statement
- Assumptions
- Architecture overview
- Component design
- Data model
- API contracts
- Risk considerations



 **github-actions** (bot) commented 12 minutes ago • edited ▾

MiDAS finished @GYH2KOR's task in 6m 14s — [View job](#) • [midas/issue-3768-20260219-0559](#) • [Create PR →](#)

Design Document Complete

Successfully created comprehensive design document for SS6 Health Check Endpoint feature!

Progress:

- ✓ Analyze repository structure
- ✓ Understand SS6 service architecture
- ✓ Research existing health check patterns
- ✓ Create comprehensive design document
- ✓ Commit design document
- ✓ Complete



Design Document Summary

File Location: `docs/design/ss6-health-check-endpoint-design.md`

Document Size: 42 KB / ~1,500 lines

Commit: `fe0e48a931afeb34b2aa73ecddbde009ca242cca`



Document Contents

Core Sections

1. **Executive Summary** - Business context and objectives
2. **Architecture Overview** - System design with 3 Mermaid diagrams
3. **Component Design** - Detailed component architecture and responsibilities
4. **Data Models** - Pydantic models and response structures
5. **API Design** - Complete endpoint specifications with examples
6. **Infrastructure & Deployment** - Kubernetes configurations and Docker setup
7. **Security Design** - Authentication strategy and risk analysis
8. **Monitoring & Observability** - Metrics, logging, and alerting
9. **Technology Decisions** - 5 major design decisions with rationale
10. **Implementation Plan** - 3-phase rollout strategy



Technical Details

Files to Create

```
ss6-serviceapi-platform/  
├─ app/api/v1/health_management/  
│   ├── __init__.py  
│   ├── health_datamodels.py  
│   └── health_router.py  
└─ tests/unit/api/v1/health_management/  
    ├── __init__.py  
    └── test_health_router.py
```

Files to Modify

- `app/appmain.py` - Register health router
- `deployment/kubernetes/deployment.yaml` - Add probes
- `Dockerfile` - Add HEALTHCHECK instruction

Kubernetes Probe Configuration

```
livenessProbe:  
  httpGet:  
    path: /services/api/v1/health  
    port: 5001  
  initialDelaySeconds: 30  
  periodSeconds: 10  
  timeoutSeconds: 5  
  failureThreshold: 3  
  
readinessProbe:  
  httpGet:  
    path: /services/api/v1/health  
    port: 5001  
  initialDelaySeconds: 15  
  periodSeconds: 5  
  timeoutSeconds: 3  
  failureThreshold: 2
```

References Included

- FastAPI health check best practices
- Kubernetes probe documentation
- Existing MiDAS service implementations (SS1, SS4, SS7)
- Azure Load Balancer documentation

Next Steps

The design document is complete and ready for review. Implementation can proceed in 3 phases:

1. **Immediate:** Phase 1 implementation (Basic health check)
2. **Short-term:** Phase 2 implementation (Kubernetes integration)
3. **Future:** Phase 3 implementation (Enhanced observability)

Related Issue

Closes: [#3768](#)

Ask

What This Validates

Using this simple story, you can verify that the Design Agent:

- Understands repository context
 - Produces structured design documentation
 - Defines API contracts clearly
 - Documents architectural decisions
 - Maintains professional documentation format
-

Best Practices

To get high-quality design outputs:

- Provide a clear and complete problem statement.
 - Mention scale requirements explicitly (e.g., expected QPS, number of users).
 - Specify constraints (cloud provider, database restrictions, internal standards).
 - Clarify whether the system is new or an extension of an existing service.
 - Indicate integration points with other MiDAS subsystems if applicable.
-

Limitations

- The agent relies on the clarity of the issue description.
 - It does not execute code or validate runtime feasibility.
 - It assumes reasonable defaults when constraints are not specified.
 - It does not replace architecture review processes.
-

Summary



The `/design` agent enables structured, high-quality architectural documentation directly from GitHub issues. It ensures consistency, improves design clarity, and reduces ambiguity before implementation begins.

It should be used as the first step in any substantial feature or system-level change.

